EAFIT Social Network

1. Context

This practice is designed to strengthen the understanding of graphs as a data structure for modeling relationships between entities. In the course presentation on graphs, the topic is introduced through the notions of vertex, edge, adjacency, degree, connectivity, cycles, acyclicity, and the projection toward graph algorithms such as BFS, DFS, Prim, Kruskal, and Dijkstra. The guiding bibliography of our course explicitly frames this sequence from structural concepts toward algorithms in Chapter 9, Graph Algorithms, of Mark Allen Weiss, Data Structures and Algorithm Analysis in C++, Fourth Edition.
In this sense, the practice focuses on the implementation and explanation of an undirected graph, where an edge represents a mutual relationship between two vertices. This interpretation is consistent with the course explanation that, in an undirected graph, if an edge exists between A and B, then the relation is read in both directions.

2. Purpose of the practice

The purpose of this practice is to design and implement a complete program in C++ that models an undirected graph by means of an appropriate internal representation, preferably an adjacency list, since the course presentation identifies this structure as the natural representation for graph traversals and related algorithms in sparse graphs.

The solution must not be limited to a mechanical implementation. On the contrary, it must make explicit the relationship between the conceptual definition of graph and its computational representation, so that you can justify design decisions, explain the meaning of each structure used, and connect the implementation to the theoretical framework developed in the course bibliography.

3. General statement

In Universidad EAFIT, the Academic IT Office has commissioned your development team to design and implement a prototype of a campus social network for students. In this prototype, each student is represented as a node, and a mutual friendship between two students is represented as an undirected edge.

The goal of the prototype is not to build a complete production system, but to provide a clear, well-documented implementation of the underlying graph structure that will later support more advanced features, such as friend recommendations, community detection, or event diffusion across the student network.

Develop a C++ program that implements an undirected graph and allows its use as a pedagogical and computational model for representing relationships between entities.
The program must be written with:

- Class names in Spanish.
- Attribute names in Spanish.
- Method names in Spanish.
- Inline comments in Spanish.
- Clear structure, readable coding style, and coherent object-oriented design.

The submitted solution must be accompanied by an oral explanation in class in which the student justifies the representation of the graph, explains how vertices and edges are stored, and shows how the program reflects the essential ideas discussed in the course regarding undirected graphs, degree, adjacency, connectivity, and traversal.

4. Functional requirements

The program must include, at minimum, the following features:

a. Graph construction
   - Add vertices.
   - Add undirected edges.
   - Ensure that when an edge is inserted between two vertices, the connection is reflected in both adjacency lists.

b. Graph representation
   - Store the graph using an adjacency-list-based structure.
   - Allow the complete graph to be displayed in a readable format.

c. Structural queries
   - Determine whether a vertex exists.
   - Determine whether two vertices are adjacent.
   - Compute the degree of a vertex.
   - Compute the total number of vertices.
   - Compute the total number of edges.

d. Traversals
   - Implement Breadth-First Search (BFS).
   - Implement Depth-First Search (DFS).

- Explain the conceptual difference between both traversals during the presentation.

5. Interpretation of the graph

- Use an example scenario with meaningful entities, such as cities and roads, people and friendships, or locations and bidirectional connections.
- Explain why the modeled relation is undirected.

6. Technical requirements

The implementation must satisfy the following technical conditions:

- The main solution must be developed in standard C++.
- The code must compile and run correctly.
- The program must contain a main execution example.
- The implementation must include classes with a coherent responsibility assignment.
- The names of classes, methods, attributes, and variables must be semantically meaningful.
- The comments must explain the purpose of each relevant section of the program.
- The program must avoid unnecessary duplication and must preserve internal consistency in the graph representation.

7. Suggested structure

Although the exact design may vary, the solution is expected to include at least one central class such as UndirectedGraph, together with the corresponding methods for insertion, query, display, and traversal.
A stronger submission may additionally include complementary abstractions such as:

- Vertex
- Edge
- Auxiliary methods for recursive DFS
- Validation methods for repeated insertion or invalid queries

8. Bibliographic articulation

According to the course presentation, the study of graphs begins with the distinction between directed and undirected graphs, continues with graph representation, degree, connectivity, cycles, and acyclicity, and then leads into graph algorithms. The slides explicitly connect this progression with Weiss's presentation in Chapter 9, where definitions and representations are introduced

before topological sort, shortest paths, minimum spanning trees, and DFS-based applications.

Therefore, the present practice should be understood not as an isolated coding exercise, but as the structural basis for later topics in the semester. A correct implementation of an undirected graph is valuable precisely because it prepares the student to understand why BFS, DFS, Prim, Kruskal, and Dijkstra require a rigorous graph representation in the first place.

9. Deliverables

Each student or team must submit the following:

a. Source code in C++ throw Interactiva Platform and OnlineGDB.
b. A short written explanation in English describing:
   - the selected graph representation,
   - the meaning of an undirected edge in the proposed scenario,
   - how degree is computed,
   - how BFS and DFS operate at a conceptual level.
c. In-class explanation or demonstration of the solution.

10. Evaluation rubric

The practice will be evaluated according to the following detailed rubric:

Descriptors	Grade			
	Excellent (4.6 - 5.0)	Good (4.0 - 4.5)	Acceptable (3.0 - 3.9)	Insufficient (0 - 2.9)
Conceptual understanding of undirected graphs (6%)	Defines graph, vertex, edge, adjacency, and degree with precision; clearly explains the difference between directed and undirected graphs; accurately justifies why the chosen scenario is modeled as undirected.	Shows an overall correct understanding, with minor imprecision in terminology or explanation.	Understands the central idea but presents weak justification or partial confusion in concepts.	Shows major confusion regarding the nature of graphs or the meaning of undirected edges.

Data structure design (4,5%)	Uses a coherent design, appropriate abstraction, and a representation aligned with the problem; class and method responsibilities are well defined.	Design is correct and functional, though not especially elegant.	Representation works, but design is weak, rigid, or poorly organized.	Representation is inappropriate, inconsistent, or conceptually incorrect.
Vertex and edge management (6%)	Vertex and edge insertion works correctly in all normal cases; undirected symmetry is properly preserved.	Core insertion works, with minor omissions such as limited validation.	Basic insertion works, but behavior is incomplete or fragile.	Edge management is incorrect, especially if the graph behaves as directed by mistake.
Structural operations (6%)	All required structural queries are correct and consistent with the internal representation.	Most operations are correct, with only minor inaccuracies.	Some operations work, but others are incomplete or weakly justified.	Several required operations are missing or incorrect.
Traversal algorithms (4,5%)	BFS and DFS are correctly implemented and clearly explained in conceptual and operational terms.	Both traversals work, though the explanation may lack depth.	Traversals are partially correct or insufficiently explained.	BFS or DFS is missing, incorrect, or cannot be explained.
Code clarity and documentation (3%)	Code is highly readable, consistently named in English, well commented, and professionally structured.	Code is understandable and mostly consistent.	Code is functional but difficult to read or weakly documented.	Code is disorganized, poorly named, or lacks meaningful comments.

Oral explanation and defense (70%)	The explanation is fluent, rigorous, and pedagogically clear; design choices are defended with confidence.	The explanation is clear overall, though not fully rigorous in every aspect.	The solution is presented, but with hesitation or weak conceptual articulation.	The presenter cannot explain the implemented solution coherently.

11. Additional recommendations for students

- Start from the conceptual question: what are the entities and what are the mutual relations between them?
- Make sure the graph truly behaves as undirected.
- Verify that the degree of a vertex matches the number of neighbors stored for that vertex.
- Use the oral presentation to connect code and theory, not merely to read code line by line.
- Keep in mind that this practice is a bridge toward later graph algorithms in the semester, particularly BFS, DFS, minimum spanning tree algorithms, and shortest-path reasoning.

12. Closing note

From the perspective of the course, understanding undirected graphs means more than memorizing a definition. It means learning how to identify vertices and edges in a real problem, deciding how the relation should be modeled, choosing a suitable representation, and interpreting the structure through adjacency, degree, connectivity, and traversal. The presentation used in class explicitly places these concepts as the conceptual basis of the graph algorithm block developed in Weiss's text.